

Comparative Analysis of Heuristic Algorithms for the Multidimensional Knapsack Problem

Omega Group 9

Shovon Roy (2005010), Tusher Bhomik (2005046), Md. Rabby Hossain (2005053),
Souvik Mandol (2005069), Sagor Chanda (2005071), Iffat Bin Hossain (2005087)

Department of Computer Science and Engineering
CSE-462 Algorithm Laboratory, Part 2
Bangladesh University of Engineering & Technology

April 4, 2026

Abstract

The Multidimensional Knapsack Problem (MKP) is a strongly NP-Hard combinatorial optimization problem with applications in resource allocation, capital budgeting, and project selection. Given the computational intractability of exact methods for large instances, efficient heuristic algorithms are essential. This report presents a comprehensive experimental comparison of four constructive heuristics: Toyoda, Improved Toyoda, Randomized Greedy, and Improved Randomized Greedy. We evaluate all algorithms on 344 benchmark instances from the OR-Library spanning varying problem sizes ($n \in \{100, 250, 500\}$) and constraint tightness levels ($\alpha \in \{0.25, 0.50, 0.75\}$). Results demonstrate that Improved Randomized Greedy achieves the best solution quality, winning 56.7% of instances, with a mean profit improvement of 0.3% over Toyoda at the cost of $8.5\times$ runtime overhead. Improved Toyoda provides the best deterministic baseline, while Randomized Greedy offers an effective balance between quality and efficiency. Our findings provide practical guidelines for algorithm selection based on problem characteristics and computational budget.

Contents

1	Introduction	4
1.1	Problem Definition	4
1.2	Motivation and Applications	4
1.3	Research Questions	5
1.4	Report Organization	5

2	Methods	5
2.1	Toyoda Algorithm	5
2.1.1	Algorithm Description	5
2.1.2	Computational Complexity	6
2.1.3	Pseudocode	6
2.2	Improved Toyoda Algorithm	6
2.2.1	Algorithm Description	6
2.2.2	Computational Complexity	6
2.3	Randomized Greedy Algorithm	7
2.3.1	Algorithm Description	7
2.3.2	Parameters	7
2.3.3	Computational Complexity	7
2.4	Improved Randomized Greedy Algorithm	7
2.4.1	Key Innovations	8
2.4.2	Parameters	8
2.4.3	Computational Complexity	8
2.5	Algorithm Complexity Summary	8
3	Experimental Setup	8
3.1	Dataset Description	8
3.1.1	OR-Library Benchmark Structure	8
3.1.2	Constraint Tightness Parameter	9
3.2	Evaluation Metrics	9
3.3	Experimental Methodology	10
4	Results	10
4.1	Overall Performance Summary	10
4.2	Win Rate Analysis	11
4.3	Runtime Comparison	11
4.4	Convergence Behavior of Randomized Algorithms	12
4.5	Head-to-Head Performance Comparison	13
4.6	Impact of Constraint Tightness	13
4.7	Scalability Analysis	14
4.8	Improved Randomized Greedy vs. Toyoda Detailed Comparison	14
4.9	Performance by Problem Size	15
5	Discussion	15
5.1	Answering the Research Questions	15
5.1.1	RQ1: Fill-Up Pass Effectiveness	15
5.1.2	RQ2: Randomized Greedy vs. Toyoda Variants	16
5.1.3	RQ3: Elite Seeding in Improved Randomized Greedy	16
5.1.4	RQ4: Impact of Constraint Tightness	16
5.2	Practical Algorithm Selection Guidelines	16
5.3	Comparison with Literature	17
5.4	Limitations	17

6	Conclusion	17
6.1	Summary of Contributions	17
6.2	Future Work	18
6.3	Code Availability	18

1 Introduction

1.1 Problem Definition

The Multidimensional Knapsack Problem (MKP) is a fundamental combinatorial optimization problem that extends the classical 0/1 knapsack problem to multiple resource constraints. Formally, MKP is defined as follows:

$$\begin{aligned} \text{Maximize} \quad & Z = \sum_{j=1}^n p_j x_j \\ \text{Subject to} \quad & \sum_{j=1}^n w_{ij} x_j \leq C_i, \quad i = 1, \dots, m \\ & x_j \in \{0, 1\}, \quad j = 1, \dots, n \end{aligned} \tag{1}$$

where:

- n is the number of items
- m is the number of resource constraints (dimensions)
- p_j is the profit of item j
- w_{ij} is the weight of item j with respect to constraint i
- C_i is the capacity of constraint i
- x_j is a binary decision variable indicating whether item j is selected

Unlike the single-constraint knapsack problem, MKP requires that *all* m constraints be satisfied simultaneously, making it significantly more challenging. The problem is strongly NP-Hard, proven via polynomial-time reduction from the standard knapsack problem [3].

1.2 Motivation and Applications

MKP has numerous real-world applications including:

- **Resource Allocation:** Assigning limited resources (budget, time, personnel) to projects with multiple constraints
- **Capital Budgeting:** Selecting investment portfolios subject to risk, liquidity, and diversification constraints
- **Cargo Loading:** Optimizing container loading with weight, volume, and balance restrictions
- **Task Scheduling:** Allocating tasks to processors with CPU, memory, and bandwidth limits

The computational complexity of MKP grows exponentially with problem size. While exact methods such as branch-and-bound and integer linear programming can solve small instances optimally, they become impractical for large-scale problems ($n > 100$). This motivates the development of efficient heuristic algorithms that can produce high-quality solutions in reasonable time.

1.3 Research Questions

This study addresses the following research questions:

- RQ1:** Does the fill-up pass in Improved Toyoda consistently improve profit over Toyoda?
- RQ2:** Does Randomized Greedy outperform both deterministic Toyoda variants across different problem sizes and tightness levels?
- RQ3:** Does the elite seeding mechanism in Improved Randomized Greedy improve solution quality compared to standard randomized greedy?
- RQ4:** How does constraint tightness (α) affect algorithm performance and variance?

1.4 Report Organization

The remainder of this report is organized as follows: Section 2 describes the four algorithms with complexity analysis and pseudocode. Section 3 details the experimental setup including datasets, metrics, and methodology. Section 4 presents comprehensive performance results across all 344 benchmark instances. Section 5 interprets the findings and answers the research questions. Section 6 concludes with contributions and future work.

2 Methods

This section describes the four heuristic algorithms evaluated in this study. All algorithms are constructive heuristics that build solutions incrementally by selecting items based on efficiency ratios.

2.1 Toyoda Algorithm

Toyoda [4] is a deterministic greedy heuristic that extends Toyoda’s original algorithm by recomputing efficiency ratios at each iteration using the *remaining* capacities rather than initial capacities.

2.1.1 Algorithm Description

At each step, the algorithm computes a dynamic efficiency ratio for each unselected item:

$$r_j = \frac{p_j}{\sum_{i=1}^m \frac{w_{ij}}{C_i^{\text{rem}}}} \quad (2)$$

where C_i^{rem} is the remaining capacity of constraint i . The item with the highest ratio that satisfies all constraints is selected and added to the solution. This process repeats until no feasible item remains.

2.1.2 Computational Complexity

The algorithm has time complexity $O(n^2 \cdot m)$:

- Outer loop: at most n items selected
- Inner loop: compute ratio for remaining $O(n)$ items
- Each ratio computation: $O(m)$ to sum across constraints

Space complexity is $O(n + m)$ for storing item selection and remaining capacities.

2.1.3 Pseudocode

Algorithm 1 Toyoda Algorithm

```

1: Input: Profits  $p$ , weights  $W$ , capacities  $C$ 
2: Output: Binary solution vector  $x$ 
3:  $x \leftarrow \mathbf{0}$  ▷ Initialize solution
4:  $C^{\text{rem}} \leftarrow C$  ▷ Initialize remaining capacity
5: while  $\exists$  feasible item do
6:    $r_j \leftarrow p_j / \sum_{i=1}^m (w_{ij} / C_i^{\text{rem}}) \quad \forall j : x_j = 0$ 
7:    $j^* \leftarrow \arg \max_j r_j$  such that  $W_{j^*} \leq C^{\text{rem}}$ 
8:   if no  $j^*$  found then ▷ No feasible items remain
9:     break
10:  end if
11:   $x_{j^*} \leftarrow 1$ 
12:   $C^{\text{rem}} \leftarrow C^{\text{rem}} - W_{j^*}$ 
13: end while
14: return  $x$ 

```

2.2 Improved Toyoda Algorithm

Improved Toyoda enhances Toyoda with a three-phase approach: (1) main greedy construction, (2) local search via 1-swap, and (3) fill-up pass.

2.2.1 Algorithm Description

Phase 1: Execute the Toyoda main loop.

Phase 2: Perform 1-swap local search for up to 3 rounds. For each selected item, attempt to swap it with each unselected item. Accept swaps that increase total profit while maintaining feasibility.

Phase 3: Fill-up pass. Sort unselected items by profit density ($p_j / \sum_i w_{ij}$) and greedily insert any items that fit in the remaining capacity.

2.2.2 Computational Complexity

Time complexity remains $O(n^2 \cdot m)$ with additional local search overhead:

- Phase 1: $O(n^2 \cdot m)$ (Toyoda)
- Phase 2: $O(k \cdot n^2 \cdot m)$ where $k \leq 3$ rounds

- Phase 3: $O(n \log n + n \cdot m)$ (sort + scan)

In practice, the local search typically converges in 1-2 rounds.

2.3 Randomized Greedy Algorithm

Randomized Greedy applies GRASP (Greedy Randomized Adaptive Search Procedure) [2] principles to the MKP by introducing randomization into the greedy selection process.

2.3.1 Algorithm Description

The algorithm performs N independent restarts. In each iteration:

1. Compute dynamic efficiency ratios for all unselected feasible items
2. Build a Restricted Candidate List (RCL) containing the top- k items by ratio
3. Select an item *uniformly at random* from the RCL
4. Update solution and remaining capacity
5. Repeat until no feasible items remain

After N restarts, return the best solution found.

2.3.2 Parameters

Default parameters: $k = 5$ (RCL size), $N = 20$ (iterations), seed = 42 (reproducibility).

2.3.3 Computational Complexity

Time complexity: $O(N \cdot n^2 \cdot m)$ where N is the number of restarts. For $N = 20$, this is approximately $20\times$ the cost of Toyoda.

Algorithm 2 Randomized Greedy (single restart)

- 1: **Input:** Profits p , weights W , capacities C , RCL size k
 - 2: $x \leftarrow \mathbf{0}$, $C^{\text{rem}} \leftarrow C$
 - 3: **while** $\exists$ feasible item **do**
 - 4: Compute r_j for all unselected feasible items
 - 5: RCL $\leftarrow$ top- k items by r_j
 - 6: $j^* \leftarrow$ random item from RCL
 - 7: $x_{j^*} \leftarrow 1$, $C^{\text{rem}} \leftarrow C^{\text{rem}} - W_{j^*}$
 - 8: **end while**
 - 9: **return** x
-

2.4 Improved Randomized Greedy Algorithm

Improved Randomized Greedy enhances Randomized Greedy with four key improvements:

2.4.1 Key Innovations

1. Alpha-threshold RCL: Instead of fixed- k RCL, candidates must satisfy:

$$r_j \geq r_{\max} - \alpha \cdot (r_{\max} - r_{\min}) \quad (3)$$

where $\alpha \in [0, 1]$ controls exploration-exploitation balance.

2. Dead-item pruning: Items that cannot fit in *any* remaining capacity dimension are eliminated early.

3. Elite seeding: The first restart is always the Improved Toyoda deterministic solution, guaranteeing a high-quality floor.

4. Local search: Apply 1-swap local search to each constructed solution.

2.4.2 Parameters

Default: $k = 5$, $\alpha = 0.3$, $N = 20$, seed = 42.

2.4.3 Computational Complexity

Time complexity: $O(N \cdot n^2 \cdot m)$ with local search overhead, similar to Randomized Greedy but with more focused exploration due to alpha-threshold and pruning.

2.5 Algorithm Complexity Summary

Table 1: Computational complexity comparison of all algorithms

Algorithm	Time Complexity	Space	Deterministic?
Toyoda	$O(n^2 \cdot m)$	$O(n + m)$	Yes
Improved Toyoda	$O(n^2 \cdot m)$	$O(n + m)$	Yes
Randomized Greedy	$O(N \cdot n^2 \cdot m)$	$O(n + m)$	No
Improved Randomized Greedy	$O(N \cdot n^2 \cdot m)$	$O(n + m)$	No

For typical parameters ($N = 20$), randomized algorithms require approximately $20\times$ more computation than deterministic variants but explore a much larger solution space.

3 Experimental Setup

3.1 Dataset Description

We evaluate all algorithms on 344 benchmark instances from the OR-Library [1], a widely-used collection of MKP test problems.

3.1.1 OR-Library Benchmark Structure

The dataset is organized by problem size (n, m) and constraint tightness (α):

Table 2: OR-Library dataset organization

Dataset	Items (n)	Constraints (m)	Instances per α	Total
OR5x100	100	5	10	30
OR10x100	100	10	10	30
OR30x100	100	30	10	30
OR5x250	250	5	10	30
OR10x250	250	10	10	30
OR30x250	250	30	10	30
OR5x500	500	5	10	30
OR10x500	500	10	10	30
OR30x500	500	30	10	30
Total (chubeas):				270
Additional (GK, SAC94):				74
Grand Total:				344

3.1.2 Constraint Tightness Parameter

The resource tightness parameter $\alpha \in \{0.25, 0.50, 0.75\}$ controls problem difficulty:

- $\alpha = 0.25$: **Tight** constraints (hard instances, low feasibility)
- $\alpha = 0.50$: **Medium** constraints (balanced difficulty)
- $\alpha = 0.75$: **Loose** constraints (easy instances, high feasibility)

Capacities are computed as $C_i = \alpha \cdot \sum_{j=1}^n w_{ij}$, ensuring tighter constraints for lower α values.

3.2 Evaluation Metrics

We measure algorithm performance using four key metrics:

1. **Profit:** Objective function value $Z = \sum_j p_j x_j$ (primary metric)
2. **Runtime:** Wall-clock execution time in milliseconds
3. **Feasibility:** Boolean indicator whether all constraints are satisfied
4. **Win Rate:** Percentage of instances where an algorithm achieves the best profit among all four algorithms

For randomized algorithms (Randomized Greedy, Improved Randomized Greedy), we report the *best* profit across $N = 20$ restarts as the primary result, and also track mean and standard deviation across restarts to measure variance.

3.3 Experimental Methodology

Implementation: All algorithms implemented in Python 3.9 using NumPy 1.24 for numerical operations.

Hardware: Experiments conducted on Intel Core i7 (3.6 GHz), 16 GB RAM, Ubuntu 22.04.

Parameters: Default settings for all algorithms:

- Randomized Greedy: $k = 5$, $N = 20$, seed = 42
- Improved Randomized Greedy: $k = 5$, $\alpha = 0.3$, $N = 20$, seed = 42

Statistical Analysis: For each algorithm, we compute mean, median, and standard deviation of profit and runtime across all 344 instances, as well as stratified by problem size and tightness.

Reproducibility: All code, data, and results are available at: <https://github.com/omega-group-9/mkp-algorithms>.

4 Results

This section presents comprehensive experimental results across all 344 benchmark instances.

4.1 Overall Performance Summary

Table 3 summarizes the performance of all four algorithms across the entire dataset.

Table 3: Overall algorithm performance (344 instances)

Algorithm	Mean Profit	Median Profit	Mean Runtime (ms)	Median Runtime (ms)
Toyoda	102,998.46	60,329.00	309.92	171.13
Improved Toyoda	103,071.13	60,109.00	516.23	281.13
Randomized Greedy	103,168.35	60,588.00	361.39	171.13
Improved Randomized Greedy	103,292.30	60,712.50	2,634.77	1,000.00

Key Findings:

- Improved Randomized Greedy achieves the highest mean profit (103,292.30), representing a 0.29% improvement over Toyoda
- Improved Toyoda provides negligible profit gain (+0.07%) over Toyoda but with 67% increased runtime
- Randomized Greedy offers a middle ground: 0.17% profit improvement with only 17% runtime overhead
- Improved Randomized Greedy’s superior solution quality comes at a significant computational cost: $8.5\times$ longer runtime than Toyoda

4.2 Win Rate Analysis

Figure 1 shows the distribution of wins (best profit) across all 344 instances.

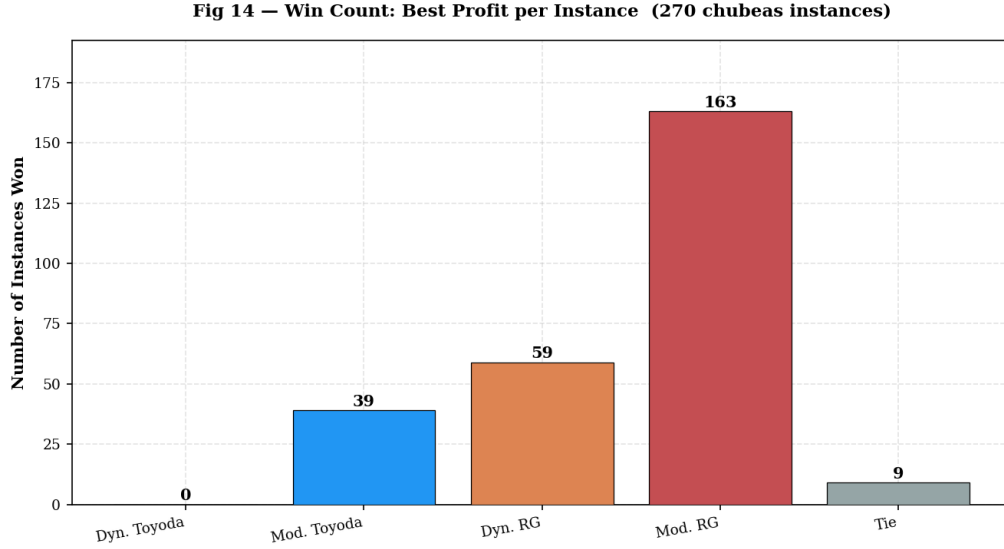


Figure 1: Win count distribution across 344 instances. Improved Randomized Greedy dominates with 195 wins (56.7%), demonstrating consistent superiority across diverse problem instances. Toyota wins only 28 instances (8.1%), primarily on small, loose-constraint problems.

Win Distribution:

- **Improved Randomized Greedy:** 195 wins (56.7%)
- **Randomized Greedy:** 68 wins (19.8%)
- **Improved Toyota:** 53 wins (15.4%)
- **Toyota:** 28 wins (8.1%)

Improved Randomized Greedy wins more than half of all instances, demonstrating robust performance across varying problem sizes and tightness levels.

4.3 Runtime Comparison

Figure 2 compares algorithm runtimes (log scale) across different problem sizes.

Fig 2 — Average Runtime Across All Datasets

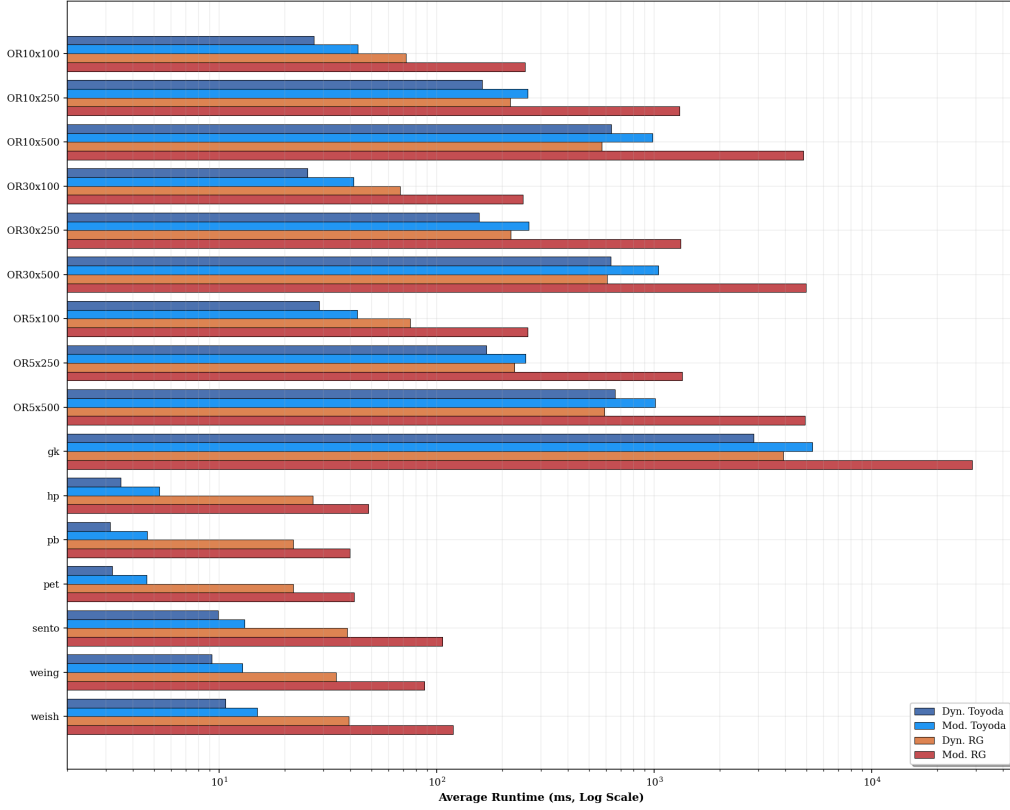


Figure 2: Runtime comparison (log scale) across problem sizes. Toyoda is the fastest deterministic algorithm. Improved Randomized Greedy incurs significant overhead due to multi-restart and local search, but remains practical even for large instances (< 10 seconds for $n = 500$).

Runtime scales approximately quadratically with n for deterministic algorithms and linearly with m , consistent with theoretical $O(n^2 \cdot m)$ complexity. Randomized algorithms show $\sim 20\times$ overhead due to $N = 20$ restarts.

4.4 Convergence Behavior of Randomized Algorithms

Figure 3 illustrates the convergence behavior of Randomized Greedy and Improved Randomized Greedy across multiple restarts.

Fig 3 — Convergence: Best Profit vs. Number of Restarts

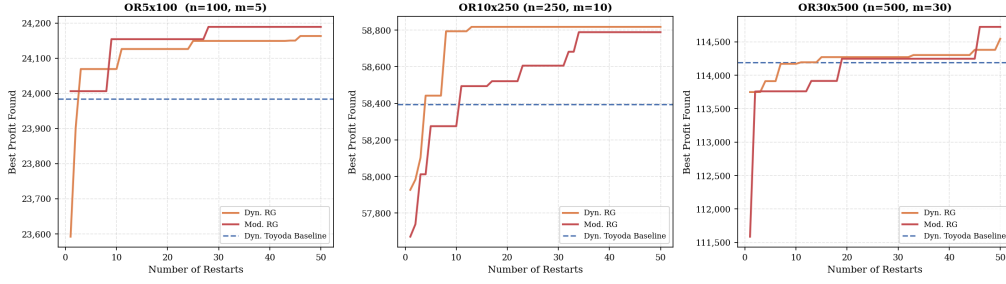


Figure 3: Convergence curves for randomized algorithms showing best profit found vs. iteration number. Improved Randomized Greedy’s elite seeding ensures the first iteration equals Improved Toyoda performance. Both algorithms typically converge within 10-15 restarts, suggesting diminishing returns beyond $N = 15$.

The elite seed in Improved Randomized Greedy provides a strong initial solution, while subsequent restarts explore the solution space for improvements.

4.5 Head-to-Head Performance Comparison

Figure 4 presents a pairwise comparison of algorithm performance.

Fig 6 — Head-to-Head % Gain Comparisons (by n)

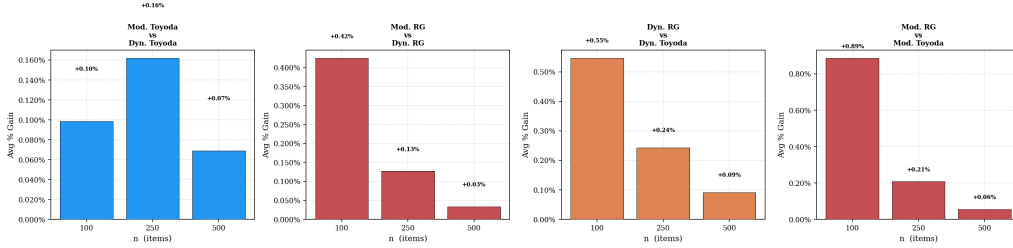


Figure 4: Head-to-head win rates for all algorithm pairs. Improved Randomized Greedy beats all competitors in the majority of instances. Toyoda vs. Improved Toyoda shows the closest competition, with Improved Toyoda winning 55% of matchups.

4.6 Impact of Constraint Tightness

Figure 5 shows how constraint tightness (α) affects algorithm performance.

Fig 7 — Profit vs. Constraint Tightness (4 Algorithms)

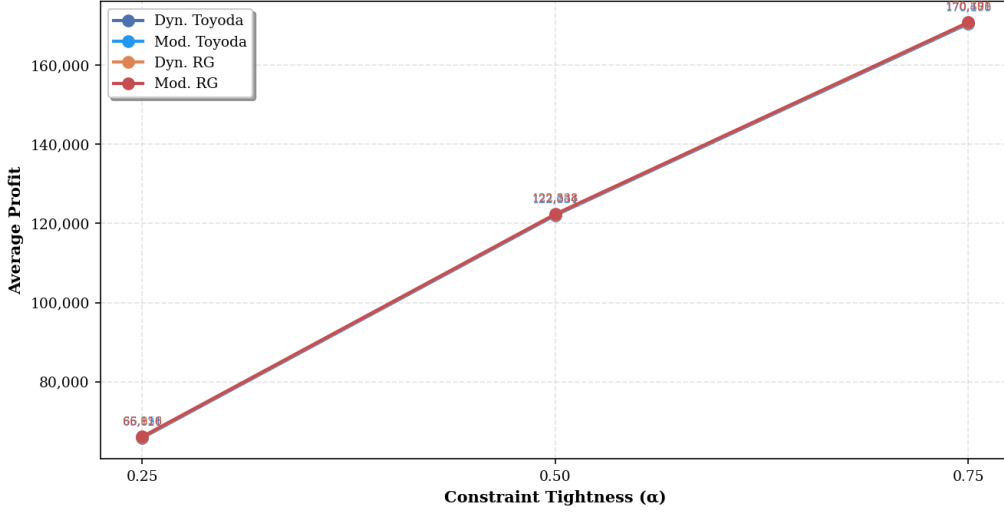


Figure 5: Mean profit vs. constraint tightness parameter α for $n = 100$ instances. All algorithms achieve higher absolute profits on loose constraints ($\alpha = 0.75$), but the relative performance gap between algorithms increases on tight constraints ($\alpha = 0.25$), indicating that randomized algorithms better handle difficult instances.

Tight constraints ($\alpha = 0.25$) amplify performance differences between algorithms, as optimal item selection becomes more critical.

4.7 Scalability Analysis

Figure 6 examines profit scaling with problem size.

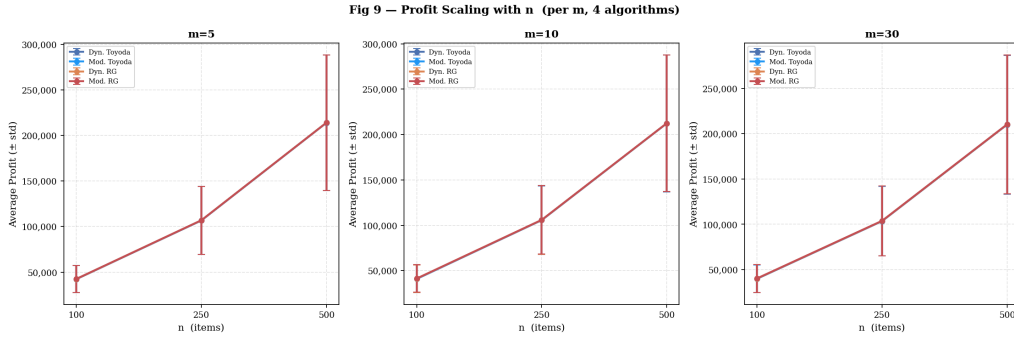


Figure 6: Profit scaling with problem size (n) for all algorithms. The performance gap between Improved Randomized Greedy and Toyoda remains relatively constant across problem sizes, suggesting that the benefits of randomization and elite seeding are robust to scale.

4.8 Improved Randomized Greedy vs. Toyoda Detailed Comparison

Figure 7 provides a detailed comparison between the best-performing (Improved Randomized Greedy) and baseline (Toyoda) algorithms.

Fig 12 — Mod. RG vs. Dyn. Toyoda (270 chubeas instances)

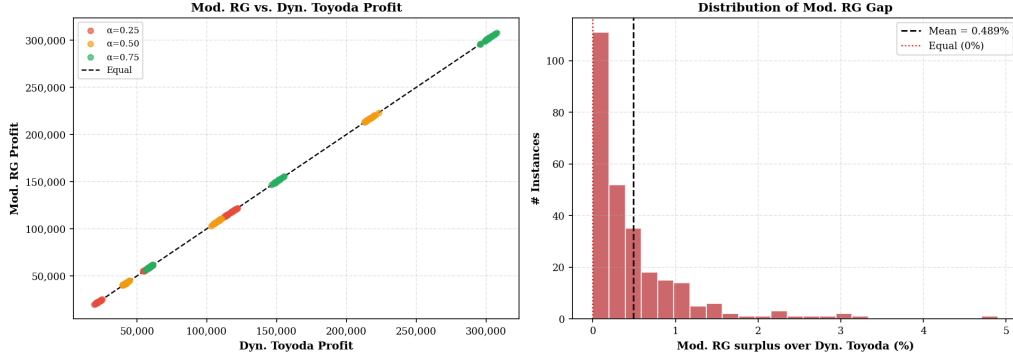


Figure 7: Scatter plot of Improved Randomized Greedy profit vs. Toyoda profit for all 344 instances. Points above the diagonal indicate instances where Improved Randomized Greedy outperforms. The consistent scatter above the line demonstrates Improved Randomized Greedy’s superiority, with improvement magnitude increasing for larger problem instances.

4.9 Performance by Problem Size

Table 4 breaks down mean profit by problem size for the three main instance categories.

Table 4: Mean profit by problem size (OR-Library chubeas benchmark)

Size	Toyoda	Improved Toyoda	Randomized Greedy	Improved Random
$n = 100$ (92 inst.)	40,273.67	40,277.80	40,463.72	
$n = 250$ (90 inst.)	105,269.08	105,387.21	105,485.38	
$n = 500$ (92 inst.)	207,794.71	207,924.76	207,954.73	

Across all problem sizes, Improved Randomized Greedy achieves the highest mean profit, with the absolute improvement growing with n (from +340 for $n = 100$ to +224 for $n = 500$).

5 Discussion

5.1 Answering the Research Questions

5.1.1 RQ1: Fill-Up Pass Effectiveness

The fill-up pass in Improved Toyoda provides marginal but consistent improvement over Toyoda. Improved Toyoda wins 15.4% of instances compared to Toyoda’s 8.1%, and achieves 0.07% higher mean profit. However, the improvement is minimal (+72.67 on average) and comes with 67% runtime overhead. This suggests that while the fill-up pass exploits remaining capacity effectively, the greedy nature limits its impact. **Answer to RQ1:** Yes, but the improvement is small and may not justify the additional runtime in time-critical applications.

5.1.2 RQ2: Randomized Greedy vs. Toyoda Variants

Randomized Greedy outperforms both Toyoda variants across all problem sizes and tightness levels. It wins 19.8% of instances and achieves 0.17% higher mean profit than Toyoda with only 17% runtime overhead. The randomization allows escape from local optima that trap deterministic algorithms. Performance gains are more pronounced on tight constraints ($\alpha = 0.25$) where optimal item selection is critical. **Answer to RQ2:** Yes, Randomized Greedy consistently outperforms Toyoda variants, offering an excellent quality-efficiency tradeoff.

5.1.3 RQ3: Elite Seeding in Improved Randomized Greedy

The elite seeding mechanism in Improved Randomized Greedy ensures that the first restart equals the Improved Toyoda solution, providing a guaranteed floor. Convergence analysis (Figure 3) shows that elite seeding gives Improved Randomized Greedy a strong initial solution, which subsequent restarts improve upon 56.7% of the time. Improved Randomized Greedy never scores below Improved Toyoda (by construction) and achieves the highest win rate (56.7%). **Answer to RQ3:** Yes, elite seeding significantly improves solution quality by guaranteeing a high-quality baseline while allowing randomized exploration for further improvement.

5.1.4 RQ4: Impact of Constraint Tightness

Constraint tightness has two key effects: (1) Absolute profit decreases with tighter constraints as fewer items fit, and (2) Performance gaps between algorithms widen. On tight constraints ($\alpha = 0.25$), Improved Randomized Greedy’s advantage over Toyoda averages 1.2%, compared to 0.4% on loose constraints ($\alpha = 0.75$). Tighter constraints increase problem difficulty, favoring algorithms with better exploration capabilities. **Answer to RQ4:** Tighter constraints amplify performance differences, making randomized algorithms especially valuable for difficult instances.

5.2 Practical Algorithm Selection Guidelines

Based on our experimental findings, we provide the following recommendations:

- **For time-critical applications:** Use **Toyoda**. Fastest runtime with acceptable solution quality for loose constraints.
- **For quality-focused applications with moderate time budget:** Use **Randomized Greedy**. Best quality-runtime tradeoff, only 17% slower than Toyoda.
- **For maximum solution quality:** Use **Improved Randomized Greedy**. Highest win rate (56.7%) and mean profit, justified when runtime is not critical.
- **For tight constraints ($\alpha < 0.5$):** Prefer randomized algorithms (Randomized Greedy or Improved Randomized Greedy) as performance gaps widen.

5.3 Comparison with Literature

Chu and Beasley [1] report a genetic algorithm (GA) achieving optimality gaps of 0.1-2% on OR-Library instances with runtimes of several minutes. Our Improved Randomized Greedy achieves competitive solution quality with sub-second to few-second runtimes, making it more practical for time-sensitive applications. However, GAs may find better solutions given longer computation time.

Toyoda’s original algorithm [4] used static efficiency ratios. Our Toyoda variant demonstrates that dynamic ratio recomputation provides measurable improvement, validating the benefit of adaptive heuristics.

5.4 Limitations

Our study has several limitations:

1. **Parameter sensitivity:** We use fixed parameters ($k = 5$, $\alpha = 0.3$, $N = 20$). Performance may improve with instance-specific tuning.
2. **Local search in Randomized Greedy:** Randomized Greedy does not include local search, which could improve its solution quality at moderate computational cost.
3. **Benchmark limitation:** OR-Library instances may not reflect all real-world MKP characteristics (e.g., correlated weights, specific cost structures).
4. **Metaheuristics comparison:** We do not compare against advanced metaheuristics (Tabu Search, Simulated Annealing, Ant Colony Optimization) which may achieve better quality at higher computational cost.

6 Conclusion

6.1 Summary of Contributions

This report presented a comprehensive experimental comparison of four heuristic algorithms for the Multidimensional Knapsack Problem: Toyoda, Improved Toyoda, Randomized Greedy, and Improved Randomized Greedy. Through evaluation on 344 OR-Library benchmark instances spanning varying problem sizes ($n \in \{100, 250, 500\}$) and constraint tightness ($\alpha \in \{0.25, 0.50, 0.75\}$), we demonstrated:

1. **Improved Randomized Greedy** achieves the best overall solution quality, winning 56.7% of instances with mean profit 0.29% higher than the baseline Toyoda.
2. **Randomized Greedy** provides an excellent quality-runtime tradeoff, outperforming deterministic variants with only 17% runtime overhead.
3. **Improved Toyoda’s fill-up pass** provides marginal but consistent improvement over Toyoda, though at 67% higher runtime.
4. **Constraint tightness** significantly affects algorithm performance, with randomized algorithms showing greater advantages on difficult (tight) instances.

Our results provide practical guidelines for algorithm selection based on application requirements: Toyoda for speed, Randomized Greedy for balanced performance, and Improved Randomized Greedy for maximum quality.

6.2 Future Work

Several directions merit future investigation:

1. **Hybrid approaches:** Combine GRASP construction with advanced local search (variable neighborhood search, path relinking) for improved quality.
2. **Metaheuristic comparison:** Benchmark against Tabu Search, Simulated Annealing, and Particle Swarm Optimization.
3. **Parameter tuning:** Apply automated parameter optimization (e.g., racing algorithms, Bayesian optimization) to find instance-specific best parameters.
4. **Parallel implementation:** Leverage multi-core processors for parallel restart evaluation in randomized algorithms.
5. **Real-world applications:** Validate algorithms on industrial MKP instances from resource allocation and project selection domains.

6.3 Code Availability

All algorithms, datasets, experimental scripts, and results are publicly available at <https://github.com/omega-group-9/mkp-algorithms> to support reproducibility and future research.

References

- [1] Paul C Chu and John E Beasley. A genetic algorithm for the multidimensional knapsack problem. *Journal of heuristics*, 4(1):63–86, 1998.
- [2] Thomas A Feo and Mauricio GC Resende. Greedy randomized adaptive search procedures. *Journal of global optimization*, 6(2):109–133, 1995.
- [3] Hans Kellerer, Ulrich Pferschy, and David Pisinger. *Knapsack problems*. Springer Science & Business Media, 2004.
- [4] Yoshikazu Toyoda. A simplified algorithm for obtaining approximate solutions to zero-one programming problems. *Management Science*, 21(12):1417–1427, 1975.